Socket Programming

Mudumbai Venkata Srivatsav

March 10, 2026

Motivation

TCP/IP Model

Application Layer
Transport Layer (TCP / UDP)
Internet Layer (IP)
Network Access Layer

- Applications running on different hosts need to communicate over a network.
- Communication between hosts is handled by transport protocols such as TCP and UDP.
- However, applications cannot directly access transport layer protocols.
- The operating system provides an interface called a **socket**.
- **Socket programming** allows applications to send and receive data using transport layer services.

Motivation(Contd)

Consider two programs running on different machines in a network.

For example:

- A client requesting data from a server
- Multiple users connecting to a service
- Distributed applications exchanging information

However, processes running on different machines cannot directly share memory or communicate using normal inter-process communication mechanisms.

Core Question

How can two processes running on different machines exchange data reliably?

What is a Socket?

A socket is a **software abstraction** that represents an endpoint of communication between processes.
Each connection is uniquely identified by

$$(\textit{IP Address},\ \textit{Port Number})$$

Example

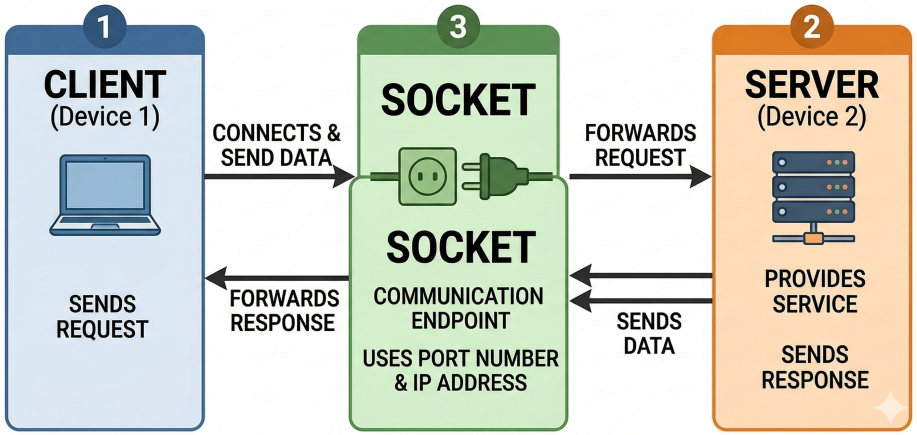
- Server IP : 192.168.1.20
- Port : 5000

A socket allows programs to:

- establish connections
- transmit data
- receive data
- terminate communication

Client–Server Architecture

Socket based communication follows a client–server model



Client

- Initiates communication
- Sends requests to the server
- Receives responses

Server

- Waits for client connections
- Processes client requests
- Sends responses

Transport Layer Services

The transport layer provides end-to-end communication between applications running on different hosts.

Two major transport protocols used in socket programming:

- TCP (Transmission Control Protocol)
- UDP (User Datagram Protocol)

Applications choose the transport protocol depending on reliability, latency, and application requirements.

TCP Communication Model

Transmission Control Protocol (TCP) provides reliable, connection-oriented communication between applications.

Key properties:

- Connection-oriented communication (connection established before data transfer)
- Reliable delivery using acknowledgements and retransmissions
- Ordered data delivery using sequence numbers
- Congestion and flow control mechanisms

TCP provides a **reliable byte-stream service**, ensuring that data sent by a client arrives correctly and in order at the server.

UDP Communication Model

User Datagram Protocol (UDP) provides a simple, connectionless communication mechanism.

Key properties:

- Connectionless communication (no connection setup required)
- Best-effort delivery (packets may be lost)
- No built-in ordering or retransmission
- Very small protocol overhead

UDP delivers independent packets called **datagrams**. If ordering or reliability is required, it must be implemented by the application layer.

UDP is commonly used in latency-sensitive applications such as video streaming and online gaming.

Basic Socket Programming Functions

Socket programming is performed using system calls provided by the operating system. These functions allow applications to create communication endpoints and exchange data over a network.

Function	**Purpose**
socket()	Create a communication endpoint
bind()	Associate socket with an IP address and port
listen()	Enable server to accept connections
accept()	Accept a client connection
connect()	Initiate connection to server
send()/recv()	Exchange data between client and server
close()	Terminate the connection

Important Socket Function Signatures

Function	Signature
socket()	`int socket(int domain, int type, int protocol);`
bind()	`int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`
listen()	`int listen(int sockfd, int backlog);`
accept()	`int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);`
connect()	`int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`
send()	`ssize_t send(int sockfd, const void *buf, size_t len, int flags);`
recv()	`ssize_t recv(int sockfd, void *buf, size_t len, int flags);`

Important Socket Parameters

Parameter	**Meaning**
domain	Specifies the address family (e.g., AF_INET for IPv4 communication)
type	Specifies the communication type (SOCK_STREAM for TCP, SOCK_DGRAM for UDP)
sockfd	Socket identifier returned by `socket()` and used in subsequent socket operations
addr	Structure containing the destination or local IP address and port
buf	Memory buffer used to store data being sent or received

TCP Client–Server Workflow

Client	**Server**
socket()	socket()

TCP Client–Server Workflow

Client	**Server**
socket()	socket()
	bind()

TCP Client–Server Workflow

Client	Server
socket()	socket()
	bind()
	listen()

TCP Client–Server Workflow

Client		**Server**
socket()		socket()
		bind()
		listen()
connect()	$\rightarrow$	accept()

TCP Client–Server Workflow

Client		Server
socket()		socket()
		bind()
		listen()
connect()	$\rightarrow$	accept()
send()	$\rightarrow$	recv()

TCP Client–Server Workflow

Client		Server
socket()		socket()
		bind()
		listen()
connect()	$\rightarrow$	accept()
send()	$\rightarrow$	recv()
recv()	$\leftarrow$	send()

TCP Client–Server Workflow

Client		Server
socket()		socket()
		bind()
		listen()
connect()	$\rightarrow$	accept()
send()	$\rightarrow$	recv()
recv()	$\leftarrow$	send()
close()		close()

UDP Socket Workflow

Client		**Server**
socket()		socket()
		bind()
sendto()	$\rightarrow$	recvfrom()
recvfrom()	$\leftarrow$	sendto()

Demo 1: Multi-Client TCP Echo Server

Objective

Demonstrate TCP-based client–server communication with multiple clients using socket programming.

Task

- Implement a TCP server listening on port 5000
- Three clients connect to the server
- Each client sends a distinct message:
  - Client 1: `"Message from Client 1"`
  - Client 2: `"Message from Client 2"`
  - Client 3: `"Message from Client 3"`
- The server echoes the received message back to the corresponding client

Demo 2: TCP vs UDP Performance Comparison

Objective

Evaluate the performance differences between TCP and UDP communication using socket programming.

Task

- Implement TCP and UDP echo servers listening on port 5000
- A client sends **1000 messages**: `"Performance Test Message"`
- The server echoes each message back to the client

Measurements

- Compute the **average Round Trip Time (RTT)**
- Measure **packet loss** (messages without response)

Hint: Use **struct timeval start,end** with **gettimeofday()**

$$RTT\,(ms) = (end.tv_sec - start.tv_sec) \times 1000 + (end.tv_usec - start.tv_usec)/1000$$

Demo 3: Server Scheduling

Objective

Study how scheduling policies affect server performance when multiple clients submit jobs concurrently.

Task

- Implement a TCP server listening on port **5000**.
- Launch at least **5 concurrent clients**. Each client submits at least **3 job requests**.
- Each job includes:
  - processing time: **1–5 seconds**
  - priority: **1–5** (higher value = higher priority)
- Implement both **FCFS** and **Priority Scheduling** at the server and compare their performance using the metrics **waiting time** and **response time**.